实验五：分词器、注意力与 Transformer 组件

- 实验五：分词器、注意力与 Transformer 组件
  - 概述
    - 实现内容
  - 环境配置
  - 任务总览与推荐完成顺序
  - Task 1：字节级 BPE 与分词器训练
    - 文本如何变成 token
    - BPE 的合并规则从何而来
    - 步骤 1.1：`BPETokenizer._merge`
    - 步骤 1.2：`BPETokenizer.train`
    - 步骤 1.3：`BPETokenizer._encode_ordinary`
    - 检查点 A
  - Task 2：构造滑动窗口训练样本
    - 步骤 2.1：`sliding_window`
    - 检查点 B
  - Task 3：数值稳定的 softmax 与缩放点积注意力
    - 注意力在算什么
    - 步骤 3.1：`softmax`
    - 步骤 3.2：`scaled_dot_product_attention`
    - Task 3 中间检查
  - Task 4：因果掩码、多头注意力与 RoPE
    - 因果掩码：禁止访问未来位置
    - 步骤 4.1：`causal_mask`
    - 多头注意力：并行地看多种模式
    - 步骤 4.2：`MultiHeadAttention.forward`
    - RoPE：用旋转编码位置
    - 步骤 4.3：`apply_rope`
    - 检查点 C
  - Task 5：LayerNorm、前馈网络与 Transformer block
    - 步骤 5.1：`LayerNorm.forward`
    - 步骤 5.2：`gelu`
    - 步骤 5.3：`FeedForward.forward`
    - 步骤 5.4：`TransformerBlock.forward`
    - 检查点 D：全部通过
  - 可视化与实验报告
    - 实验报告要求
  - 提供的代码（无需修改）
  - 评分标准
    - 提交要求
  - 参考资料

概述

自注意力（self-attention）让序列中的每个位置根据内容计算与其他位置的相关性，并据此汇聚信息。它是 Transformer 的核心机制[1]。

不过，模型不能直接处理字符串。文本首先需要被编码成 token 序列，再由向量表示进入注意力层。因此，本实验从文本的字节表示出发，依次实现分词器、语言模型样本构造、注意力机制，以及一个可堆叠的 Transformer block。

本次实验的目标是：仅使用 NumPy 与 Python 标准库，从零训练一个字节级 BPE 分词器，并实现因果多头注意力、旋转位置编码（RoPE）与 pre-norm Transformer block 的前向计算。本实验不涉及神经网络组件的反向传播或完整语言模型的训练，重点是理解各组件的计算原理、张量形状与连接方式。

实现内容

按依赖关系，你需要依次完成以下五个部分：

顺序	部分	内容	所在文件
①	Task 1	字节级 BPE：相邻对合并 → 词表训练 → 文本编解码	`src/tokenizer.py`

顺序	部分	内容	所在文件
②	Task 2	语言模型数据：构造滑动窗口输入与下一 token 预测目标	`src/tokenizer.py`
③	Task 3	注意力基础：数值稳定的 softmax → 缩放点积注意力	`src/attention.py`
④	Task 4	因果掩码 → 并行多头注意力，以及独立的 RoPE 旋转操作	`src/attention.py`
⑤	Task 5	LayerNorm / GELU → 逐位置前馈网络 → pre-norm Transformer block	`src/transformer.py`

环境配置

请首先安装 `uv` Python 项目管理工具（如果之前的实验已经安装过，可跳过）：

```
# On macOS and Linux.
curl -LsSf https://astral.sh/uv/install.sh | sh
```

```
# On Windows.
powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex"
```

安装完成后，在项目根目录下执行：

```
uv sync
```

该命令会创建 Python 虚拟环境，并安装本实验所需的 `numpy`、`matplotlib`、`pytest` 和 `tiktoken`。若使用 IDE，可将 Python 解释器设置为该虚拟环境中的解释器。为减少路径兼容性问题，建议将项目放在仅含 ASCII 字符的路径下。

接着运行测试，检查环境是否配置成功：

```
uv run pytest
```

若输出类似以下内容，则说明环境配置成功：

```
FFFFFFFFFFF.FFF..FFF..FFFFFFFFFFFFFFFFFFFFFFFFFFFFF                      [100%]
========================== short test summary info ===========================
FAILED tests/test_attention.py::test_softmax_rows_sum_to_one - NotImplementedError
FAILED tests/test_attention.py::test_softmax_is_numerically_stable - NotImplementedError
...
46 failed, 5 passed in 0.41s
```

其中，5 个通过的测试对应已提供的辅助代码，其余测试将在完成相应任务后通过。

调试时可按文件或测试名筛选；需要查看报错位置时，加上 `--tb=short`：

```
uv run pytest tests/test_tokenizer.py
uv run pytest tests/test_attention.py -k softmax
uv run pytest tests/test_transformer.py --tb=short
```

任务总览与推荐完成顺序

下面这张表是整个实验的「地图」。第 3 列列出实现依赖；建议从上到下推进，每完成一组函数就运行对应测试，再进入下一步。

步骤	函数/方法	依赖	完成后运行的验证
1.1	`BPETokenizer._merge`	无	3 个 `test_merge` 测试
1.2	`BPETokenizer.train`	1.1	3 个 `test_train` 测试
1.3	`BPETokenizer._encode_ordinary`	1.1、1.2	4 个 `encode` / `compression` 测试
1	检查点 A：BPE 全部通过	—	`uv run pytest tests/test_tokenizer.py -k "not sliding_window"` （1

步骤	函数/方法	依赖	完成后运行的验证
2.1	`sliding_window`	无， 可直接用整数序列测试	6 个 `sliding_window` 测试
2	检查点 B： 分词与样本构造全部通过	检查点 A、2.1	`uv run pytest tests/test_tokenizer.py`（18/18）
3.1	`softmax`	无	4 个 `softmax` 测试
3.2	`scaled_dot_product_attention`	3.1	3 个不使用掩码的 `sdpa` 测试
4.1	`causal_mask`	3.2 （用于验证掩码效果）	3 个因果掩码与首位置测试
4.2	`MultiHeadAttention.forward`	3.2、4.1	5 个 `mha` 测试
4.3	`apply_rope`	已提供的 `compute_rope_params`	5 个 `rope` 测试
4	检查点 C：注意力全部通过	3.1～4.3	`uv run pytest tests/test_attention.py`（20/20）
5.1	`LayerNorm.forward`	无	3 个 `layernorm` 测试
5.2	`gelu`	无	4 个 `gelu` 测试
5.3	`FeedForward.forward`	5.2	2 个 `feedforward` 测试
5.4	`TransformerBlock.forward`	4.2、5.1、5.3	4 个 `transformer_block` 测试
5	检查点 D：全部 51 例通过	检查点 B、C、5.1～ 5.4	`uv run pytest`（51/51）

Task 1：字节级 BPE 与分词器训练

文本如何变成 token

文本必须先表示为数字序列，才能作为模型的输入。本实验分两步完成这件事：

1. 文本 → 字节。将文本编码为 UTF-8 字节序列。英文 ASCII 字符占一个字节，常见汉字通常占三个字节，emoji 可能占更多字节。每个字节的取值范围都是 `0..255`，因此只用 256 个基础符号，就能表示各种语言的文本。
2. 字节 → token。若每个字节都占一个 token，序列会很长。字节级 BPE（Byte-Pair Encoding，字节对编码）反复合并高频的相邻 token 对，把常见单词或片段表示为较少的 token；未见过的组合仍能退回到单字节表示。BPE 的子词分词方法见参考资料[2]，GPT-2 使用了它的字节级变体[3]。

下面使用 GPT-2 分词器（词表大小为 50257）编码一段文本，观察单词、空格、标点、大小写与子词的切分方式：

```
import tiktoken

text = "Hello, do you like tea? <|endoftext|> In the sunlit terraces of someunknownPlace."
raw_bytes = text.encode("utf-8", errors="replace")
print("Byte count:", len(raw_bytes))

tokenizer = tiktoken.get_encoding("gpt2")
token_ids = tokenizer.encode(text, allowed_special={"<|endoftext|>"})
print("Token IDs:", token_ids)
for tid in token_ids:
    print(tid, repr(tokenizer.decode([tid])))
print("Compression ratio (bytes/token):", len(raw_bytes) / len(token_ids))
```

token 不一定是完整的单词：空格可能与后面的单词一起编码，罕见词可能被拆成多个子词，特殊 token 则可以被识别为一个整体。`tiktoken` 仅用于这里的效果对照，接下来需要从零训练自己的分词器。

BPE 的合并规则从何而来

GPT-2 的合并规则由 OpenAI 在大规模语料上预先训练。本任务使用短篇语料 `data/the-verdict.txt`，训练一个总词表大小为 1000 的简化字节级 BPE 分词器。算法如下：

```
普通词表 ← 256 个字节（id 0..255）
序列 ← 训练语料的字节 id 序列
为特殊 token 预留词表容量
重复，直到普通词表达到容量上限，或序列中不再有相邻对：
    1. 统计当前序列中所有相邻 token 对的出现频率
    2. 选出频率最高的一对 (a, b)
    3. 分配新 id，记录规则 (a, b) -> new_id
       新 token 的字节内容为 vocab[a] + vocab[b]
    4. 从左到右，把所有不重叠的 (a, b) 替换成 new_id
在已学得的普通 token 之后，为特殊 token 分配 id
```

每次合并都会缩短训练序列，并在词表中增加一个更长的 token。例如，对 `abcabc` 训练两次合并：先把 `(a, b)` 合成 `ab`，序列变为 `[ab, c, ab, c]`；再把 `(ab, c)` 合成 `abc`，序列变为 `[abc, abc]`。

`BPETokenizer` 中已经提供了两个辅助函数：

- `_pair_stats`：统计相邻 token 对的频率，统计时允许重叠。
- `_most_frequent_pair`：选择频率最高的一对；若频率相同，选择 token id 字典序较小的一对，使结果可复现。

步骤 1.1：`BPETokenizer._merge`

目标：实现一次相邻对替换，返回新的 token id 列表，不修改输入列表。

函数签名：

```python
def _merge(self, ids: list[int], pair: tuple[int, int], new_id: int) -> list[int]:
    # ids: 当前 token id 序列
    # pair: 要替换的相邻对 (a, b)
    # 返回替换后的新列表
```

实现要求：从左到右扫描，命中 `pair` 时输出 `new_id` 并跳过两个位置，否则保留当前 id。合并不重叠，例如 `[a, a, a]` 合并 `(a, a)` 后得到 `[new_id, a]`。

验证：

```
uv run pytest tests/test_tokenizer.py -k test_merge
```

应看到 3 passed。

步骤 1.2：`BPETokenizer.train`

目标：按照上述算法学习合并规则，并建立普通 token 与特殊 token 的映射。

函数签名：

```python
def train(self, text: str, vocab_size: int, allowed_special: frozenset = frozenset()) -> None:
    # text: 训练语料，假定不包含待保留的特殊 token
    # vocab_size: 总词表容量，包含 256 个字节、合并得到的 token 与特殊 token
    # 结果保存在实例的词表与合并规则中，不返回编码序列
```

实现要求：

1. 先为 `allowed_special` 中的特殊 token 预留容量。若 `vocab_size < 256 + len(allowed_special)`，抛出 `ValueError`。
2. 从 256 个字节 token 开始，调用 `_pair_stats` 与 `_most_frequent_pair` 选择相邻对，记录到 `self.merges`，更新 `self.vocab`，再调用 `_merge` 重写序列。
3. 普通词表达到容量上限时停止；若序列中已没有相邻对，也应提前停止，不必强行填满词表。
4. 在已学得的普通 token 之后为特殊 token 分配 id，并维护 `self.special_tokens` 与 `self.inverse_special_tokens`。为保证顺序稳定，可先对特殊 token 字符串排序。

`self.vocab` 保存普通 token 的 `id → bytes` 映射，特殊 token 单独存储；总词表大小为 `len(self.vocab) + len(self.special_tokens)`。

验证：

```
uv run pytest tests/test_tokenizer.py -k test_train
```

应看到 3 passed。

步骤 1.3：`BPETokenizer._encode_ordinary`

目标：使用已经学得的规则，将不含特殊 token 的普通文本编码为 token id 序列。

函数签名：

```
def _encode_ordinary(self, text: str) -> list[int]:
    # text: 不含特殊 token 的文本
    # 返回 token id 列表
```

实现要求：先将文本转换为 UTF-8 字节 id。之后反复检查当前序列中的相邻对：在所有有合并规则的相邻对中，选择最早学得的规则（对应的新 id 最小），调用 `_merge` 执行替换；直到不存在可合并的相邻对。

已提供的 `encode` 通过 `allowed_special` 控制特殊 token 的识别，`decode` 负责将 token 还原为文本。

验证：

```
uv run pytest tests/test_tokenizer.py -k "encode or compression"
```

应看到 4 passed。

检查点 A

完成以上 3 个方法后，运行：

```
uv run pytest tests/test_tokenizer.py -k "not sliding_window"
```

应看到 12 passed。

接下来，可以在实际语料上训练自己的分词器：

```
from pathlib import Path
from src.tokenizer import BPETokenizer, compression_ratio

corpus = Path("data/the-verdict.txt").read_text(encoding="utf-8")
tokenizer = BPETokenizer()
tokenizer.train(corpus, vocab_size=1000, allowed_special=frozenset({"<|endoftext|>"}))

text = corpus[:200]
ids = tokenizer.encode(text)
print("Token count:", len(ids))
print("Round trip:", tokenizer.decode(ids) == text)
print("Compression ratio (bytes/token):", compression_ratio(text, tokenizer))
```

`compression_ratio` 返回 UTF-8 字节数除以 token 数（bytes/token），数值越大，表示同一段文本所需的 token 越少。

Task 2：构造滑动窗口训练样本

语言模型的训练目标是预测下一个 token。给定一段文本，标签已经包含在文本本身中，不需要另行人工标注：当前位置的输入对应下一个位置的 token。

取一个长度为 `max_length + 1` 的窗口，前 `max_length` 个 token 作为输入，后 `max_length` 个 token 作为目标，就得到一个训练样本。相邻窗口的起点间隔为 `stride`：

```
token 序列:   [t0, t1, t2, t3, t4, t5, ...]
窗口 (max_length=4):
  inputs :    [t0, t1, t2, t3]
  targets:    [t1, t2, t3, t4]
```

步骤 2.1：`sliding_window`

目标：使用 NumPy 构造滑动窗口的输入与目标数组。

函数签名：

```
def sliding_window(token_ids, max_length: int, stride: int):
    # token_ids: 长度为 T 的一维 token id 序列
    # max_length: 每个输入、目标序列的长度
    # stride: 相邻样本起点之间的间隔
    # 返回 (inputs, targets)，均为 (num_samples, max_length) 的 np.int64 数组
```

实现要求：

- 对起点 `s`，输入为 `token_ids[s:s + max_length]`，目标为 `token_ids[s + 1:s + max_length + 1]`。
- 只保留完整样本，即 `s + max_length < T`；末尾不足一个窗口的部分直接舍弃，不做填充。
- 若 `T <= max_length`，无法构造一个完整样本，应抛出 `ValueError`。

当 `stride < max_length` 时，相邻输入样本重叠；当 `stride == max_length` 时，相邻输入样本首尾相接、不重叠。

验证：

```
uv run pytest tests/test_tokenizer.py -k sliding_window
```

应看到 6 passed。

检查点 B

```
uv run pytest tests/test_tokenizer.py
```

应看到 18 passed。

Task 3：数值稳定的 softmax 与缩放点积注意力

注意力在算什么

自注意力让每个位置根据内容汇聚其他位置的信息。对输入向量做三组不同的线性投影，得到查询（Query，Q）、键（Key，K）和值（Value，V）：

- Query：当前位置要寻找什么信息。
- Key：各位置提供什么信息，用来与 Query 比较相关性。
- Value：各位置实际提供、最终被加权汇总的内容。

Query 与各个 Key 的点积形成相关性分数；经过 softmax 得到权重，再用权重对 Value 加权求和。矩阵形式为[1]：

$$\mathrm{Attention}(Q, K, V) = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right)V$$

其中 d 是 Query 与 Key 的特征维度。$QK^\top$ 的每一行对应一个查询位置，每一列对应一个键位置，因此 softmax 沿最后一维进行：对每个查询，决定向各个键分配多少权重。

步骤 3.1：`softmax`

目标：实现沿指定轴计算的、数值稳定的 softmax。

softmax 将实数分数 s_c 变为概率：

$$p_c = \frac{e^{s_c}}{\sum_k e^{s_k}}$$

直接计算指数时，较大的分数会导致上溢。例如，`np.exp(1000)` 会得到 `inf`。标准做法是先减去归一化轴上的最大值 $m = \max_k s_k$：

$$\frac{e^{s_c - m}}{\sum_k e^{s_k - m}} = \frac{e^{s_c}e^{-m}}{\sum_k e^{s_k}e^{-m}} = \frac{e^{s_c}}{\sum_k e^{s_k}}.$$

减去同一个常数不会改变结果，却能使指数的输入都不大于 0，避免上溢。这就是 softmax 的平移不变性。

函数签名：

```
def softmax(x, axis=-1):
    # x: 任意形状的分数数组
    # 返回与 x 同形状的数组，沿 axis 非负且和为 1
```

实现提示：求最大值与求和时使用 `keepdims=True`，以便将结果广播回原数组。

验证：

```
uv run pytest tests/test_attention.py -k softmax
```

应看到 4 passed，包括大数值下的稳定性与平移不变性测试。

步骤 3.2：`scaled_dot_product_attention`

目标：实现上面的注意力公式，并返回上下文向量与注意力权重。

函数签名：

```
def scaled_dot_product_attention(q, k, v, mask=None):
    # q: (..., seq_q, d)
    # k: (..., seq_k, d)
    # v: (..., seq_k, d_v)
    # mask: 可选的加性掩码，可广播到 (..., seq_q, seq_k)
    # 返回 (context: (..., seq_q, d_v), weights: (..., seq_q, seq_k))
```

实现要求：

1. 计算 $QK^\top/\sqrt{d}$，其中转置只交换 `k` 的最后两个轴。
2. 若 `mask` 不为 `None`，将其加到分数上，再调用 `softmax`。
3. 用得到的权重对 `v` 加权求和，返回 `(context, weights)`。

计算作用于最后两维，以同时支持 `(seq, d)` 和 `(batch, heads, seq, d)` 等输入。

验证：先运行不依赖因果掩码的测试：

```
uv run pytest tests/test_attention.py -k "sdpa and not (causal or first_position)"
```

应看到 3 passed。其余两个 `sdpa` 测试在步骤 4.1 完成后验证。

Task 3 中间检查

```
uv run pytest tests/test_attention.py -k "(softmax or sdpa) and not (causal or first_position)"
```

应看到 7 passed。

Task 4：因果掩码、多头注意力与 RoPE

因果掩码：禁止访问未来位置

Task 2 中，输入位置 i 的目标是原序列的第 $i+1$ 个 token。训练时虽然会一次输入整个序列，但位置 i 不能读取 i 之后的内容，否则就能直接看到自己的预测目标，发生标签泄漏。

为禁止当前位置访问未来信息，需要在注意力分数上加一个因果掩码（causal mask）：

$$M_{ij} = \begin{cases} 0, & j \le i, \\ -\infty, & j > i. \end{cases}$$

未来位置的分数变为 $-\infty$，经过 softmax 后权重恰好为 0。主对角线保留，表示当前位置可以关注自己，也保证每行至少有一个可见位置，不会出现整行都被屏蔽的情况。

步骤 4.1：`causal_mask`

目标：构造可直接加到注意力分数上的因果掩码。

函数签名：

```
def causal_mask(seq_len):
    # 返回 (seq_len, seq_len) 浮点数组
    # 主对角线及以下为 0，严格上三角为 -np.inf
```

验证：

```
uv run pytest tests/test_attention.py -k "causal or first_position"
```

应看到 3 passed。

多头注意力：并行地看多种模式

单个注意力头使用一组 Q/K/V 投影衡量 token 之间的相关性。多头注意力（Multi-Head Attention，MHA）将 `d_out` 维投影结果划分为 `num_heads` 份，每份维度为 `head_dim = d_out // num_heads`。各头独立计算注意力，结果拼接后再经过输出投影，不同头因而可以表示不同类型的依赖关系。

拆分头数不等于复制同一组特征。Q/K/V 投影矩阵的不同列对应不同头的参数；拆分只是把这些特征整理成适合并行计算的形状：

```
投影后的 Q / K / V: (batch, seq, d_out)
  reshape   → (batch, seq, num_heads, head_dim)
  transpose → (batch, num_heads, seq, head_dim)
  attention → (batch, num_heads, seq, head_dim)
  transpose → (batch, seq, num_heads, head_dim)
  reshape   → (batch, seq, d_out)
  @ out_proj → (batch, seq, d_out)
```

Task 3 的注意力函数支持四维输入，可以并行计算所有头。

步骤 4.2：`MultiHeadAttention.forward`

目标：组装带因果掩码的并行多头自注意力。

函数签名：

```
def forward(self, x, return_weights=False):
    # x: (batch, seq_len, d_in)，seq_len <= context_length
    # return_weights=False：返回 output，形状 (batch, seq_len, d_out)
    # return_weights=True：返回 (output, weights)
    # weights: (batch, num_heads, seq_len, seq_len)
```

实现要求：

1. 使用 `self.W_query`、`self.W_key`、`self.W_value` 完成三组线性投影。
2. 将最后一维拆分为多个头，调整为 `(batch, num_heads, seq_len, head_dim)`。
3. 按当前 `seq_len` 构造因果掩码，调用 `scaled_dot_product_attention`，让所有头并行计算。
4. 调用已提供的 `_attention_dropout(weights, self.dropout, self.training, self.rng)`，对注意力权重施加 dropout，再与 Value 相乘得到各头输出。
5. 拼接各头，右乘 `self.out_proj`；若 `return_weights=True`，同时返回施加 dropout 后的权重。

attention dropout 将部分权重置零，并将保留项除以 `1 - dropout`，因此处理后的每行权重不一定和为 1。默认 `training=False` 时不启用。

验证：

```
uv run pytest tests/test_attention.py -k mha
```

应看到 5 passed，其中包含并行实现与逐头计算的结果对比。

RoPE：用旋转编码位置

注意力根据内容计算相关性，但语言的顺序同样重要。RoPE（Rotary Position Embedding，旋转位置编码）[4] 的做法不是直接把位置向量加到输入上，而是根据位置旋转 Query 与 Key 的特征；Value 不做这一旋转。

对每一对特征，将其看作二维向量，在位置 m 旋转 $m\theta_i$ 角。不同特征对使用不同的角速度 θ_i，从而同时表示不同尺度的位置变化。

本实验采用前后两半配对（rotate-half）的布局。设 `head_dim = 2h`，第 i 对是 (x_i, x_{i+h})。若写成 $x = [a, b]$，其中 a、b 各占一半，则旋转为：

$$R_m([a, b]) = [a \odot \cos(m\theta) - b \odot \sin(m\theta),\ b \odot \cos(m\theta) + a \odot \sin(m\theta)].$$

其中 $\odot$ 表示逐元素相乘。旋转保持向量长度不变，位置 0 的旋转则是恒等变换。更重要的是，对于同一对向量 q, k，有

$$R_m(q) \cdot R_n(k) = q \cdot R_{n-m}(k).$$

也就是说，旋转后的点积通过相对位置 $n - m$ 为注意力分数引入位置信息。

步骤 4.3：`apply_rope`

目标：使用预计算的 cos/sin 表，对输入的最后一维按位置进行旋转。本实验将 RoPE 作为独立函数验证，不接入多头注意力或 Transformer block。

函数签名：

```
def apply_rope(x, cos, sin):
    # x: (..., seq_len, head_dim)，head_dim 为偶数
    # cos, sin: (context_length, head_dim)，由 compute_rope_params 提供
    # context_length >= seq_len；使用表中的前 seq_len 行
    # 返回与 x 同形状的数组
```

`compute_rope_params` 已按前后两半配对的约定生成角度表。实现时利用广播支持任意前导维度，如 `(batch, heads, seq_len, head_dim)`。

验证：

```
uv run pytest tests/test_attention.py -k rope
```

应看到 5 passed，验证模长保持、位置 0 为恒等变换，以及相对位置性。

检查点 C

```
uv run pytest tests/test_attention.py
```

应看到 20 passed。

Task 5：LayerNorm、前馈网络与 Transformer block

注意力负责在位置之间交换信息，但一个 Transformer block 还需要逐位置的非线性变换，以及让深层网络更容易训练的归一化与残差连接。本任务把这些组件逐一实现，再与 Task 4 的多头注意力组装起来。

步骤 5.1：`LayerNorm.forward`

目标：沿每个位置的最后一维进行 LayerNorm[5]，再施加可学习的增益与偏置：

$$\mathrm{LayerNorm}(x) = \mathrm{scale} \odot \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} + \mathrm{shift}, \qquad \mu = \frac{1}{d}\sum_c x_c, \quad \sigma^2 = \frac{1}{d}\sum_c (x_c - \mu)^2.$$

对 `(batch, seq, emb_dim)` 输入而言，每个样本、每个位置都独立计算自己的均值与方差，不跨 batch，也不跨位置。这与沿批次维统计的 BatchNorm 不同，因此 LayerNorm 不依赖 batch 大小，也不需要维护运行均值。

函数签名：

```
def forward(self, x):
    # x: 最后一维长度为 emb_dim 的数组
    # self.scale、self.shift: (emb_dim,)，已初始化为 1 与 0
    # 返回与 x 同形状的数组
```

实现要求：使用总体方差（`ddof=0`），将 `self.eps` 加在平方根内部；求均值、方差时保留最后一维，以便广播。

验证：

```
uv run pytest tests/test_transformer.py -k layernorm
```

应看到 3 passed。

步骤 5.2：`gelu`

目标：实现 GELU（Gaussian Error Linear Unit）激活函数的 tanh 近似[6]，这也是 GPT-2 前馈网络采用的形式：

$$\mathrm{GELU}(x) \approx 0.5x\left(1+\tanh\left(\sqrt{\tfrac{2}{\pi}}\left(x+0.044715x^3\right)\right)\right).$$

与 ReLU 在负半轴上直接截断为 0 不同，GELU 是光滑的，在 0 左侧附近会取较小的负值；当 x 很大时接近 x，很小时接近 0。

函数签名：

```
def gelu(x):
    # x: 任意形状的数组
    # 返回逐元素计算后的数组，形状不变
```

验证：

```
uv run pytest tests/test_transformer.py -k gelu
```

应看到 4 passed。

步骤 5.3：`FeedForward.forward`

目标：实现逐位置前馈网络（position-wise feed-forward network）：

$$\mathrm{FeedForward}(x) = \mathrm{GELU}(xW_1 + b_1)W_2 + b_2.$$

它先将每个位置从 `emb_dim` 维扩张到 `4 * emb_dim` 维，经过非线性激活后再投影回来。所有位置共享同一组参数，但计算相互独立——在位置之间交换信息是注意力的职责，不是前馈网络的职责。

函数签名：

```
def forward(self, x):
    # x: 最后一维长度为 emb_dim 的数组
    # W1: (emb_dim, 4 * emb_dim), b1: (4 * emb_dim,)
    # W2: (4 * emb_dim, emb_dim), b2: (emb_dim,)
    # 返回与 x 同形状的数组
```

验证：

```
uv run pytest tests/test_transformer.py -k feedforward
```

应看到 2 passed。

步骤 5.4：`TransformerBlock.forward`

目标：将上述组件组装为一个 pre-norm（先归一化）的 Transformer block。关闭 dropout 时，计算为：

$$\begin{aligned} u &= x + \mathrm{Attention}(\mathrm{LayerNorm}_1(x)), \\ y &= u + \mathrm{FeedForward}(\mathrm{LayerNorm}_2(u)). \end{aligned}$$

残差连接为每个子层提供一条恒等通路，使子层可以学习对输入的增量修正，也有助于深层网络中的梯度传播。如果两个子层的输出都为零，那么整个 block 就应退化为恒等映射，即 `block(x) == x`。

函数签名：

```
def forward(self, x):
    # x: (batch, seq_len, emb_dim)
    # 已提供 self.norm1、self.att、self.norm2、self.ff
    # 返回 (batch, seq_len, emb_dim)，形状不变
```

实现要求：

1. 对输入做 `norm1`，送入 `att`；对注意力子层的输出施加 residual dropout，再与原输入相加。
2. 对更新后的表示做 `norm2`，送入 `ff`；对前馈子层的输出施加 residual dropout，再与更新后的表示相加。
3. 两处 residual dropout 均复用 `_attention_dropout`，传入 block 自身的 `self.dropout`、`self.training` 与 `self.rng`。它作用于子层输出，区别于 Task 4 中作用于注意力权重的 attention dropout。

验证：

```
uv run pytest tests/test_transformer.py -k transformer_block
```

应看到 4 passed，验证残差通路、pre-norm 顺序与堆叠后的形状。

检查点 D：全部通过

先运行本任务的全部测试：

```
uv run pytest tests/test_transformer.py
```

应看到 13 passed。随后运行完整测试：

```
uv run pytest
```

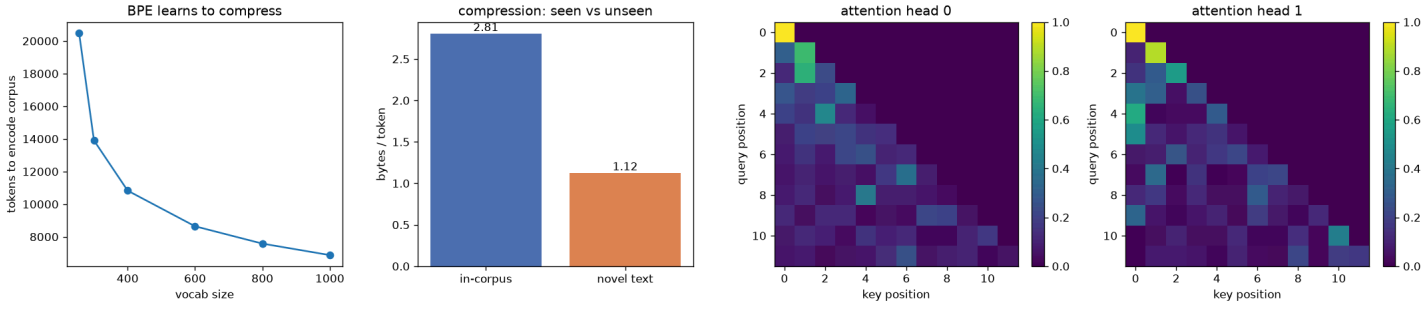
应看到 51 passed。

可视化与实验报告

全部任务完成后，在项目根目录下运行：

```
uv run python visualize.py
```

脚本会在 `data/the-verdict.txt` 上训练多个不同词表大小的 BPE 分词器，并用一个 4 头因果注意力模块处理随机向量序列，生成 `lab5.png`。参考结果如下：



四个子图从左到右展示：

1. 词表大小与语料 token 数。随着合并规则增加，同一语料通常可以用更少的 token 表示。词表为 256 且没有特殊 token 时，每个 token 就对应一个字节。
2. 语料内外文本的 bytes/token。训练语料中的常见组合更容易被合并，因此语料内文本通常具有更高的 bytes/token；图中的语料外文本选用罕见字母组合。
3. 第 0 个注意力头的权重热图。横轴为 Key 位置，纵轴为 Query 位置。严格上三角应全部为 0；第 0 行只有第 0 列为 1。
4. 第 1 个注意力头的权重热图。与前一个头比较，观察不同投影参数产生的权重分布差异。

注意力模块未经训练，热图仅用于观察因果约束与多头计算。

实验报告要求

实验报告为必交内容。请记录下表中的实验结果，回答对应问题，并附上 `visualize.py` 生成的 `lab5.png` 。报告格式不限。

实验项目	设置	需记录的结果	需回答的问题
字节与 token	GPT-2 分词器	字节数、token 数、bytes/token	空格、标点、罕见词如何切分?
BPE 从零训练	`vocab_size=1000`	语料内外 bytes/token	训练语料如何影响合并规则?
滑动窗口	不同的 `max_length` 、 `stride`	首个输入与目标、样本数	输入是否重叠? 目标是否错位一位?
稳定 softmax	如 `[1000, 1001, 1002]`	输出是否有限、是否和为 1	减最大值为何不改变结果?
因果掩码	`seq_len=6`	上三角权重、第 0 行分布	为什么必须保留主对角线?
多头注意力	`num_heads=4`	各头权重与输出形状	拆分、换轴与拼接是否互相对应?
RoPE	`head_dim=8`	模长、相同相对距离下的点积	比较时是否使用同一对 Q/K 向量?
Transformer block	pre-norm	输出形状、子层为零时的输出	残差通路是否保持恒等映射?

提供的代码（无需修改）

以下内容已经写好，直接使用：

文件或组件	内容
`src/tokenizer.py` 中的 `_pair_stats` / `_most_frequent_pair`	相邻对统计与确定性的并列处理
`src/tokenizer.py` 中的 `encode` / `decode` / `compression_ratio`	特殊 token 分派、字节还原与 bytes/token 计算
`src/attention.py` 中的 `_attention_dropout`	训练时保持期望不变的 dropout，评估时不改变输入
`src/attention.py` 中的 `compute_rope_params`	RoPE 的 cos/sin 表计算与偶数维度检查
各类的 `__init__` 与 `__call__`	参数创建、初始化，以及调用 `forward` 的接口
`data/the-verdict.txt`	BPE 训练用的文本语料
`visualize.py`	分词压缩效果与因果注意力权重的可视化脚本

评分标准

本次实验满分 100 分，全部由自动化测试决定。`uv run pytest` 共 51 例，得分按通过比例计算：

$$\text{得分} = \frac{\text{通过用例数}}{51} \times 100.$$

实验报告需按上述要求完成，并与代码一并提交。

助教以测试结果作为主要评分依据，并会对代码进行抽查。不得针对测试用例硬编码，或以其他方式绕过实际计算。

提交要求

请按要求完成实验代码，之后：

1. 将根目录下的 `.venv` 、 `.pytest_cache` 目录以及生成的 `lab5.png` 删除（如有）；保留 `data/the-verdict.txt` 等已提供的实验文件。
2. 将项目根目录（ `lab5` ）与实验报告一起以 zip 格式打包提交，压缩包命名为 `学号-姓名` 。

注意：实验过程中，除 `uv` 自动生成的文件外，只能修改 `src` 下的源文件；不得修改 `tests` 目录中的任何文件，也不得更改函数签名或已约定的输入输出接口。

提交 ddl：11月26日。

参考资料

[1] Vaswani, A., et al. (2017). Attention Is All You Need. *NeurIPS*. https://arxiv.org/abs/1706.03762 Transformer 的原始论文，介绍了缩放点积注意力、多头注意力与逐位置前馈网络。本文的 pre-norm block 与原论文使用的 post-norm 顺序不同。

[2] Sennrich, R., Haddow, B., & Birch, A. (2016). Neural Machine Translation of Rare Words with Subword Units. *ACL*. https://arxiv.org/abs/1508.07909 将 BPE 用于子词分词的代表性工作。本实验以字节而不是字符作为初始符号。

[3] Radford, A., et al. (2019). Language Models are Unsupervised Multitask Learners. *OpenAI Technical Report*. GPT-2 技术报告，涉及字节级 BPE、50257 大小的词表及语言模型结构。本实验只实现简化的字节级 BPE，不复现其完整分词流程。

[4] Su, J., et al. (2021). RoFormer: Enhanced Transformer with Rotary Position Embedding. *arXiv*. https://arxiv.org/abs/2104.09864 RoPE 的原始论文，解释了通过旋转 Q/K 将相对位置信息引入注意力分数的方法。

[5] Ba, J. L., Kiros, J. R., & Hinton, G. E. (2016). Layer Normalization. *arXiv*. https://arxiv.org/abs/1607.06450 LayerNorm 的原始论文，对应本实验沿特征维进行的归一化。

[6] Hendrycks, D., & Gimpel, K. (2016). Gaussian Error Linear Units (GELUs). *arXiv*. https://arxiv.org/abs/1606.08415 GELU 的原始论文。本实验使用其 tanh 近似形式。

[7] Raschka, S. *Build a Large Language Model (From Scratch)*. 配套代码仓库：https://github.com/rasbt/LLMs-from-scratch。分词与数据构造（ch02）、注意力（ch03）、归一化与前馈网络（ch04）均可在对应章节找到 PyTorch 版本的参照实现，供延伸阅读；本实验仍须独立使用 NumPy 完成。